

SDK REFERENCES

TypeScript SDK V2 session API (removed)

Reference for the removed V2 TypeScript Agent SDK session API, with session-based send/stream patterns for multi-turn conversations.

⚠ The V2 session API is no longer supported. TypeScript Agent SDK 0.3.142 removes `unstable_v2_createSession`, `unstable_v2_resumeSession`, `unstable_v2_prompt`, and the `SDKSession` and `SDKSessionOptions` types.

To migrate, use the `query()` API and the session options it accepts. Pass an `AsyncIterable<SDKUserMessage>` for multi-turn conversations, or `options.resume` to continue a saved session. This page is kept for reference if you maintain code on Agent SDK 0.2.x or earlier.

V2 was an experimental session API that removed the need for async generators and yield coordination. Instead of managing generator state across turns, each turn was a separate `send()` / `stream()` cycle. The API surface reduced to three concepts:

- `createSession()` / `resumeSession()` : Start or continue a conversation
- `session.send()` : Send a message
- `session.stream()` : Get the response

Installation

Agent SDK 0.2.x is the last version that includes the V2 interface. The package version jumped from 0.2.x directly to 0.3.142, so the removal version above and the install pin below describe the same boundary. To install the last V2-compatible release, pin the major and minor version:

```
npm install @anthropic-ai/claude-agent-sdk@0.2
```

ⓘ The SDK bundles a native Claude Code binary for your platform as an optional dependency, so you don't need to install Claude Code separately.

Quick start

One-shot prompt

For simple single-turn queries where you don't need to maintain a session, use

`unstable_v2_prompt()`. This example sends a math question and logs the answer:

```
import { unstable_v2_prompt } from "@anthropic-ai/claude-agent-sdk";

const result = await unstable_v2_prompt("What is 2 + 2?", {
  model: "claude-opus-4-7"
});
if (result.subtype === "success") {
  console.log(result.result);
}
```

Basic session

For interactions beyond a single prompt, create a session. V2 separates sending and streaming into distinct steps:

- `send()` dispatches your message
- `stream()` streams back the response

This explicit separation makes it easier to add logic between turns (like processing responses before sending follow-ups).

The example below creates a session, sends “Hello!” to Claude, and prints the text response. It uses `await using` (TypeScript 5.2+) to automatically close the session when the block exits. You can also call `session.close()` manually.

```
import { unstable_v2_createSession } from "@anthropic-ai/claude-agent-sdk";

await using session = unstable_v2_createSession({
  model: "claude-opus-4-7"
});

await session.send("Hello!");
for await (const msg of session.stream()) {
  // Filter for assistant messages to get human-readable output
  if (msg.type === "assistant") {
    const text = msg.message.content
      .filter((block) => block.type === "text")
      .map((block) => block.text)
      .join("");
    console.log(text);
  }
}
```

Multi-turn conversation

Sessions persist context across multiple exchanges. To continue a conversation, call `send()` again on the same session. Claude remembers the previous turns.

This example asks a math question, then asks a follow-up that references the previous answer:

```

import { unstable_v2_createSession } from "@anthropic-ai/claude-
agent-sdk";

await using session = unstable_v2_createSession({
  model: "claude-opus-4-7"
});

// Turn 1
await session.send("What is 5 + 3?");
for await (const msg of session.stream()) {
  // Filter for assistant messages to get human-readable output
  if (msg.type === "assistant") {
    const text = msg.message.content
      .filter((block) => block.type === "text")
      .map((block) => block.text)
      .join("");
    console.log(text);
  }
}

// Turn 2
await session.send("Multiply that by 2");
for await (const msg of session.stream()) {
  if (msg.type === "assistant") {
    const text = msg.message.content
      .filter((block) => block.type === "text")
      .map((block) => block.text)
      .join("");
    console.log(text);
  }
}

```

Session resume

If you have a session ID from a previous interaction, you can resume it later. This is useful for long-running workflows or when you need to persist conversations across application restarts.

This example creates a session, stores its ID, closes it, then resumes the conversation:

```

import {
  unstable_v2_createSession,
  unstable_v2_resumeSession,
  type SDKMessage
} from "@anthropic-ai/claude-agent-sdk";

// Helper to extract text from assistant messages
function getAssistantText(msg: SDKMessage): string | null {
  if (msg.type !== "assistant") return null;
  return msg.message.content
    .filter((block) => block.type === "text")
    .map((block) => block.text)
    .join("");
}

// Create initial session and have a conversation
const session = unstable_v2_createSession({
  model: "claude-opus-4-7"
});

await session.send("Remember this number: 42");

// Get the session ID from any received message
let sessionId: string | undefined;
for await (const msg of session.stream()) {
  sessionId = msg.session_id;
  const text = getAssistantText(msg);
  if (text) console.log("Initial response:", text);
}

console.log("Session ID:", sessionId);
session.close();

// Later: resume the session using the stored ID
await using resumedSession =
  unstable_v2_resumeSession(sessionId!, {
    model: "claude-opus-4-7"
  });

await resumedSession.send("What number did I ask you to
remember?");
for await (const msg of resumedSession.stream()) {
  const text = getAssistantText(msg);
  if (text) console.log("Resumed response:", text);
}

```

```
}
```

Cleanup

Sessions can be closed manually or automatically using `await using`, a TypeScript 5.2+ feature for automatic resource cleanup. If you're using an older TypeScript version or encounter compatibility issues, use manual cleanup instead.

Automatic cleanup (TypeScript 5.2+):

```
import { unstable_v2_createSession } from "@anthropic-ai/claude-agent-sdk";

await using session = unstable_v2_createSession({
  model: "claude-opus-4-7"
});
// Session closes automatically when the block exits
```

Manual cleanup:

```
import { unstable_v2_createSession } from "@anthropic-ai/claude-agent-sdk";

const session = unstable_v2_createSession({
  model: "claude-opus-4-7"
});
// ... use the session ...
session.close();
```

API reference

`unstable_v2_createSession()`

Creates a new session for multi-turn conversations.

```
function unstable_v2_createSession(options: {
  model: string;
  // Additional options supported
}): SDKSession;
```

unstable_v2_resumeSession()

Resumes an existing session by ID.

```
function unstable_v2_resumeSession(
  sessionId: string,
  options: {
    model: string;
    // Additional options supported
  }
): SDKSession;
```

unstable_v2_prompt()

One-shot convenience function for single-turn queries.

```
function unstable_v2_prompt(
  prompt: string,
  options: {
    model: string;
    // Additional options supported
  }
): Promise<SDKResultMessage>;
```

SDKSession interface

```
interface SDKSession {
  readonly sessionId: string;
  send(message: string | SDKUserMessage): Promise<void>;
  stream(): AsyncGenerator<SDKMessage, void>;
  close(): void;
}
```

Feature availability

The V2 session API does not support every V1 feature. The following require the **V1 SDK**:

- Session forking (`forkSession` option)
- Some advanced streaming input patterns

See also

- **TypeScript SDK reference (V1)** - Full V1 SDK documentation
- **SDK overview** - General SDK concepts
- **V2 examples on GitHub** - Working code examples